UNIVERSIDAD DE ANTIOQUIA
Facultad de Ingeniería

Informática II
Semestre 2026 – 2
Desafío II. 05 de octubre de 2026

Amigos para siempre, con Ude@Book

Objetivos

- Desarrollar la capacidad de análisis y solución de problemas en los estudiantes, enfrentándolos a problemáticas de la vida cotidiana.
- Verificar si el estudiante adquirió las destrezas y conocimientos fundamentales de la programación orientada a objetos en C++: abstracción, encapsulación, relaciones, diseño de diagrama de clases, funciones amigas, sobrecarga y uso de plantillas.

Si usted ha llevado un proceso disciplinado de aprendizaje a lo largo del semestre, esta es una oportunidad de demostrarlo y podrá plantear una solución satisfactoria. En caso contrario, podrá identificar sus debilidades y tomar medidas a fin de poder abordar situaciones similares a futuro.

Trate de valorar la verdadera complejidad del problema planteado; no se rinda antes de intentarlo o de plantear los posibles escenarios de solución. Se dará cuenta de que, si bien al principio le puede parecer difícil, ya ha tenido la oportunidad de enfrentarse a problemas similares. Si se toma el tiempo adecuado para analizar, el proceso de codificación será eficaz y no le tomará mucho tiempo.

Esperamos que disfrute del desafío propuesto. Lea primero todo el documento antes de comenzar y asegúrese de entender muy bien las instrucciones antes de desarrollar esta actividad evaluativa.

Revisado por los profesores Aníbal Guerra y Augusto Salazar.

Introducción

Las redes sociales han emergido en las décadas recientes con el fin de facilitar la comunicación y la creación de comunidades virtuales, logrando una amplia penetración en la sociedad actual y moviendo más de 210.000 millones de dólares anualmente (OpenAI, 2026). El objetivo de este desafío consiste en desarrollar un programa que provea las funcionalidades básicas de una red social, utilizando POO. El desarrollo se enfoca únicamente en la capa lógica, destinada a gestionar eficientemente la información de los usuarios, álbumes, fotos, grupos, amigos, entre otros.

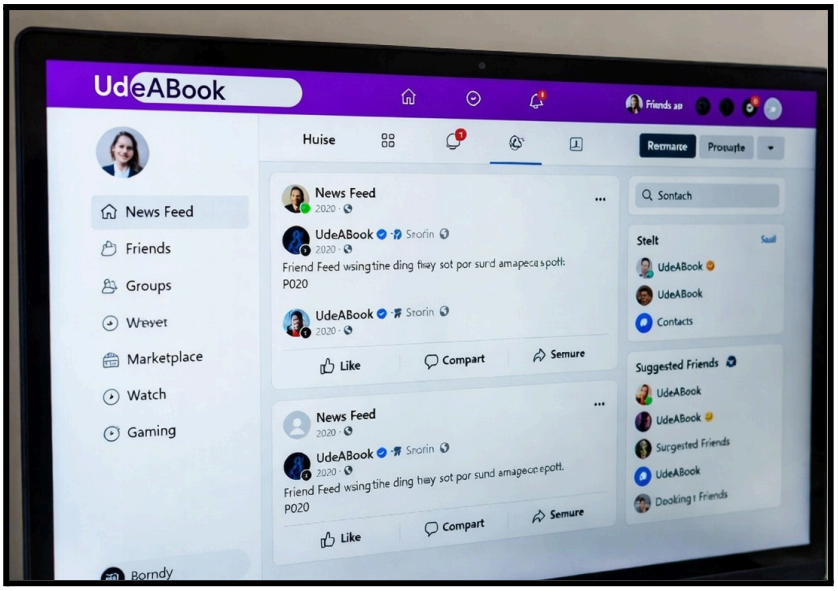


Figura 1. Ejemplo de la futura interfaz de usuario que podría tener Ude@Book (generado con Grok, Twitter)

Ude@Book

Ude@Book aspira a posicionarse como una compañía líder en el sector de las redes sociales, conformando una amplia comunidad virtual en el territorio colombiano. A futuro, se espera consolidar su operación a través de una plataforma en línea que ofrezca sus servicios a nivel mundial.

Los usuarios son la base del negocio en Ude@Book. De cada usuario en la plataforma se conoce: su nombre de usuario único, fecha de nacimiento, correo electrónico, foto de perfil (ruta), contraseña, ciudad, país y una fecha de inscripción en la plataforma. Adicionalmente, cada usuario posee:

- ★ Cero o más amigos. Los amigos son otros usuarios registrados en la plataforma con los que se ha definido mutuamente una relación de amistad.
- ★ Un conjunto de mensajes que ha enviado, ordenados por fecha de envío. Cada mensaje enviado puede tener dos estados: leído o no leído.
- ★ Un conjunto de mensajes que ha recibido, ordenados por fecha de envío.
- ★ Un conjunto de solicitudes de amistad, las cuales puede aceptar o rechazar.
- ★ Un conjunto de sugerencias de amistad. Las sugerencias de amistad muestran a los amigos de sus amigos directos, y se rankean según la cantidad de amigos en común.
- ★ Cero o más álbumes, los cuales tienen un nombre, una ruta al almacenamiento y una configuración de privacidad, pudiendo ser: públicos

(puede verlos cualquier usuario de la plataforma), privados (sólo puede verlos el dueño), o amigos: sólo para amigos. Adicionalmente, cada álbum contiene cero o más fotos.

★ De cada foto se conoce: La fecha de su captura, tamaño, ruta al archivo de en el almacenamiento y conteo de likes. Cada foto pertenece a un único álbum.

Las fotos de cada usuario se guardan en formato .png, y se encuentran almacenados en una ubicación externa expresada como una ruta absoluta en notación linux. Por ejemplo, si el usuario juanito76 tiene el álbum “Cartagena2026” y en él las fotos “Malecon2612” y “Bocagrande1”, al perfil de dicho usuario se asociarán las rutas:

/users/(*sub-ruta-X-fija*)/juanito76/albums/Cartagena2026/Malecon2612.png
/users/(*sub-ruta-X-fija*)/juanito76/albums/Cartagena2026/Bocagrande1.png
/users/(*sub-ruta-X-fija*)/juanito76/albums/profile.png

*La última ruta corresponde al almacenamiento de la foto de perfil del usuario.
**La *sub-ruta-X-fija* es la misma para todos los usuarios de la plataforma.

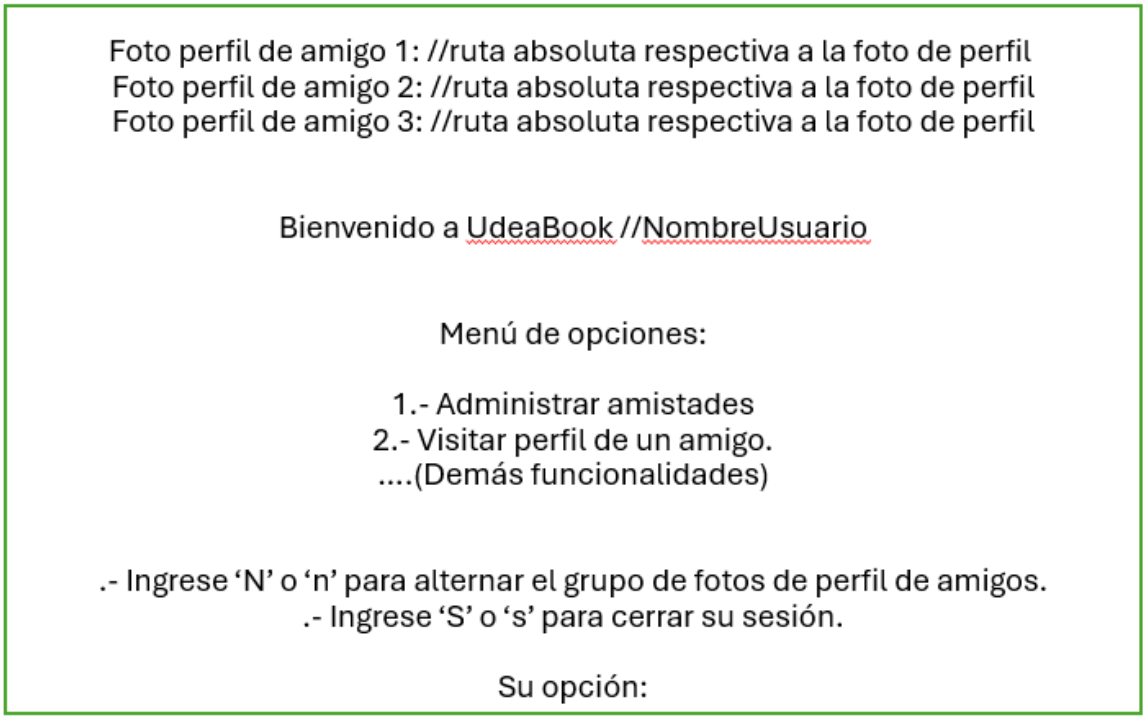


Figura 2. Ejemplo de la interfaz actual (modo consola) a mostrar a los usuarios al autenticarse (propio)

A efectos de este desafío, visualizar una foto corresponde únicamente mostrar la ruta del respectivo archivo de imagen en el almacenamiento.

Funcionalidades esenciales

Presente un menú para acceder a funcionalidades que permitan:

I - Carga / actualización de datos: Se requiere desarrollar los algoritmos necesarios para leer y actualizar los datos desde/hacia el almacenamiento permanente. Esta funcionalidad no debe aparecer en el menú presentado a los usuarios.

Los programadores deben diseñar el formato de uno o más archivos destinados a contener la información de los usuarios, amigos, solicitudes de amistad, mensajes y otros. Medite adecuadamente este asunto.

El formato de los archivos debe presentarse y explicarse adecuadamente dentro del informe de documentación a entregar. La actualización de dichos datos puede restringirse únicamente a lo que este enunciado solicita.

II - Ingreso a la plataforma y visualización de mejores amigos
Funcionalidad sencilla que permite iniciar sesión con las credenciales personales y mostrar el menú principal. Los datos requeridos para verificar el inicio de sesión se recuperan del almacenamiento permanente.

Cuando el usuario ingrese, además de presentarle el menú principal, se le deben mostrar 3 fotos de perfil seleccionadas aleatoriamente entre sus amigos. Se muestran todas a la vez y se deben acompañar de una opción que permita actualizarlas por otro grupo de 3 fotos distintas, cuidando que no se visualice la misma foto repetida en un set de 3 o con algún patrón de visualización fijo. Esta actualización de fotos de perfil de amigos sólo puede realizarse una única vez.

Para dicha visualización, no todos los amigos tienen la misma prioridad. Los usuarios con 10 o menos amigos en común tienen una probabilidad de visualización simple, los que tienen entre 11 y 20 amigos en común tienen 2.5 veces más prioridad, y los que tienen más de 20 amigos en común tienen 3.5 veces más prioridad de aparecer.

III. Administrar amistades

Permite al usuario acceder a las opciones:

a. **Enviar solicitud de amistad:** Dado un nombre de usuario, envía la solicitud de amistad a alguno de los usuarios de la plataforma, siempre que no sean amigos o no tenga pendiente una solicitud de amistad de esa persona.

b. **Aceptar solicitudes de amistad recibidas:** Permite ver la lista de solicitudes de amistad recibidas y aceptar cualquiera de ellas.

IV. Visitar perfil de un amigo

Dado el nombre de usuario de un amigo, lista sus álbumes de fotos con categoría de privacidad “amigos” o “público”. Se puede seleccionar un álbum cuyas fotos se visualizarán en modo carrusel: pudiendo cambiarse una tras otra de manera circular.

Al visualizar cada foto, el usuario tiene 4 opciones: salir del álbum, siguiente, previa, dar/quitar like a la foto.

V. Ver sugerencias de amistad

Dado un valor K entero (entre 1 y 20), esta funcionalidad muestra K sugerencias de amigos, ordenadas según la cantidad de amigos que tienen en común. Debe acompañarse de la opción de enviar solicitud a alguno de ellos.

Sugerencia investigativa: basarse en el concepto de matriz de adyacencia para representar la relación de amistad.

VI. Administrar mensajes

Esta funcionalidad permite:

a. **Enviar mensaje:** Dado el nombre de un amigo, enviarle un texto de hasta 512 caracteres.
b. **Leer nuevos mensajes:** Permite visualizar los mensajes recibidos que no han sido leídos, ordenados por fecha de envío. Se muestran todos los mensajes indicando el emisor de cada uno, luego de lo cual deben marcarse como leídos.
c. **Buscar mensajes por rango de fecha:** dada una fecha de inicio y una fecha de fin, se deben listar todos los mensajes recibidos en ese rango de fechas (ambas inclusive).

Aclaratorias técnicas:

En esta versión de la plataforma no se requieren funcionalidades para:

- Registrarse en la plataforma o eliminar la cuenta.
- Crear, cargar, editar y eliminar álbumes y fotos. Ya están creados y toda su información se carga desde el almacenamiento permanente.
- Eliminar amistades.
- Responder los mensajes leídos de forma directa.
- Cualquier otra no especificada por el cliente en este enunciado.
- En esta versión no es necesario validar que cada usuario de un único like por foto.

VII. Medición del consumo de recursos: Al finalizar la ejecución de cada una de las funcionalidades requeridas, se deben mostrar en pantalla dos métricas para cuantificar la eficiencia**:**

a. La cantidad de iteraciones requeridas (directa e indirectamente) para completar esa funcionalidad específica. Si en algún caso se invoca un componente externo, debe reportarse su uso.
b. El total de memoria que se está consumiendo en ese momento exacto de la ejecución, por causa de **TODAS** las estructuras de datos y objetos existentes en el sistema. Incluya en este valor las variables locales y parámetros por valor involucrados en la funcionalidad.

Requisitos del desarrollo

El cliente requiere un programa que provea las funcionalidades especificadas en este enunciado para ***Ude@Book***. Esto incluye la documentación del proceso de análisis, diseño y la implementación de las estructuras de datos y operaciones requeridas. La interacción con el usuario se realiza a través de la consola.

Se recomienda limitar el desarrollo a lo aquí descrito, ya que la dimensión del problema en un escenario totalmente real puede tener una complejidad superior al tiempo estipulado para la entrega. En caso de duda sobre los requerimientos, consulte con el cliente. Es muy importante que tenga presente el criterio de eficiencia para su desarrollo, ya que será parte fundamental de la evaluación.

De acuerdo con lo anterior, entre otras cosas, usted deberá:

1. **[10%]** Contextualice el problema, analícelo y diseñe el diagrama de clases correspondiente a su solución. Refleje adecuadamente las relaciones implícitas en la problemática. Recuerde utilizar la notación UML simplificada impartida en las clases teóricas. A pesar de tener una baja ponderación, la entrega de este diagrama es obligatoria y sin ella no se dará lugar a la sustentación. Considere en su modelo hacer uso adecuado de todos los conceptos estudiados, como constructores de copia, getters, setters, sobrecarga de métodos y de operadores *(sobrecargar al menos dos operadores)*, etc.

2. **[25%]** Previo a la implementación, verifique el cumplimiento del requisito de eficiencia especificado en la sección 3 del apartado “Requisitos de la entrega” de este documento. Seleccione concienzudamente los tipos y estructuras de datos que le permitirán implementar las clases planteadas en su diagrama.

3. **[10%]** Implemente las funcionalidad: “Medición del consumo de recursos”.

4. **[55%]** Presente la implementación de su programa, cuya interacción se centra en un menú que permita acceder de manera independiente a las funcionalidades requeridas en este enunciado. Considere las restricciones implícitas en la lógica del problema. Por ejemplo, no enviar solicitudes a quienes ya son amigos o a quienes actualmente nos han solicitado amistad, no enviar mensajes vacíos, entre otras.

Formalidades de la entrega

A continuación, se describen los requisitos que se deben cumplir. El incumplimiento de cualquiera de ellos implicaría alguna penalidad o que su nota sea cero.

1. Genere un informe en donde se detalle el desarrollo del proyecto, explique entre otras cosas:
   a. Análisis del problema y consideraciones para la alternativa de solución propuesta.
   b. Diagrama de clases de la solución planteada.
   c. Algoritmos esenciales debidamente intra-documentados. Describa en alto nivel la lógica de las tareas que usted definió para aquellos subprogramas cuya solución no sea trivial. No copie y pegue código en este apartado.
   d. Problemas de desarrollo que afrontó.
   e. Evolución de la solución y consideraciones a tener en cuenta en la implementación.
   f. El informe no debe ser generado por alguna herramienta de IA.

2. La solución debe ser implementada en lenguaje C++ y debe basarse en el paradigma de POO.
3. La implementación debe considerar el criterio de eficiencia y el buen diseño tanto de las estructuras de datos como de los módulos implementados. Nota: considere la utilización de referencias vs la realización de copias innecesarias sobre datos estructurados.
4. La implementación no debe utilizar contenedores de la STL, *structs* ni librerías de terceros no autorizadas que resuelvan asuntos esenciales. Las estructuras de datos usadas deben ser de creación propia y deben utilizar memoria dinámica. Puede usar la librería para generar aleatorios.
5. No usar el concepto de herencia dentro de la solución.
6. Se debe crear un repositorio público para cargar todos los archivos relacionados con la solución planteada (informe, código fuente y otros anexos).
7. Una vez cumplida la fecha de entrega no se debe hacer modificación alguna al repositorio.
8. Se deben hacer *commits* de forma regular (al menos uno cada dos días) de tal forma que se evidencie la evolución de la propuesta de solución y su implementación.
9. Se debe adjuntar un enlace de *youtube* a un video que debe incluir lo siguiente:
   a. Presentación de la solución planteada. Análisis realizado y explicación de la arquitectura del sistema (3 minutos máximo).

b. Demostración del funcionamiento del sistema. Debe incluir ejemplos demostrativos de las siguientes funcionalidades en un entorno de **mínimo 1000** usuarios registrados (6 minutos máximo):

I. Cargar datos, ingreso a la plataforma y visualización de mejores amigos (foto de perfil).
II. La consulta de 12 amigos sugeridos, y enviar solicitud de amistad al de mayor coincidencia.
III. Buscar mensajes en un rango de fecha que abarque al menos 6 semanas.
IV. Visitar el perfil de un amigo que tenga al menos 2 álbumes y 5 fotos en cada álbum (ingrese y recorra los dos álbumes).
Para cada funcionalidad, asegúrese de reportar el consumo de recursos (memoria e iteraciones requeridas).

Esta evidencia de funcionamiento debe mostrar al menos uno de los apartados mencionados (I a IV) junto con el consumo de recursos respectivo; de lo contrario, no se considerará suficiente para optar a la sustentación.

c. Explicación del código fuente. Justifique la elección de las variables y estructuras de control usadas, y destaque la ventaja que ofrecen estas en comparación con otras opciones (5 minutos máximo).
d. La duración total del video no debe exceder 15 minutos ni ser inferior a 5 minutos. No debe estar acelerado.
e. Asegúrese de que el video tenga buen sonido y que se puedan visualizar bien los componentes presentados.
f.El video no debe ser generado por alguna herramienta de IA.
g. En la elaboración del video deben participar explícitamente todos los miembros del equipo.

9. El plazo de entrega se divide en dos momentos:
   a. El día 09 de octubre para adjuntar la evidencia del proceso de análisis y diseño de la solución (Diagrama de clase incluido).
   b. El día 16 de octubre para adjuntar la evidencia del proceso de implementación.

10. En Ude@ se deben adjuntar **dos enlaces**: uno al repositorio y otro al video, nada más.
11. Para la evaluación del desafío se realizará una sustentación oral en un horario concertado con el profesor. La asistencia a la sustentación es obligatoria. Las sustentaciones que se realicen en modalidad virtual requerirán uso de cámara y el cumplimiento de condiciones de visualización por parte de los estudiantes.

Acotaciones finales

Ante la posibilidad de uso de herramientas de generación automática de soluciones se recuerda al estudiantado la responsabilidad que tiene de demostrar dominio analítico absoluto sobre la solución que entrega, así como control sobre las decisiones tomadas y el proceso de desarrollo ejecutado. Del mismo modo, se espera

un conocimiento fluido de las instrucciones y del funcionamiento de su propio código. Todo lo anterior será verificado durante la sustentación, siendo esto un factor fundamental en el proceso de calificación.

La entrega se dará por inválida en cualquiera de los siguientes casos:

- Si se omite la entrega, o se encuentra inaccesible algún enlace a: repositorio, informe, o video.
- Si la solución utiliza instrucciones de ANSI C.
- Si la solución no está debidamente diseñada usando programación modular/multi-archivos (.h, .cpp).
- Si la solución no hace uso de memoria dinámica en el core de la misma.
- Si el core de la solución se implementa delegando tareas a código de autoría externa.
- Si se utiliza el almacenamiento permanente como área para procesamiento o intercambio de datos.
- Si no se aplica POO como paradigma de la solución o si se aplica de forma trivial (menos de tres clases).
- Si se incumple cualquier otra restricción establecida en este documento.